МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Межинститутская базовая кафедра
Институт информационных технологий и кибербезопасности

Отчёт к лабораторной работе №3

Тема: **«Порождающие паттерны: Адаптер, Мост»**

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки:
09.03.04 «Программная
инженерия»

Астраков Борис Александрович

_____

Проверил:

Ассистент МИБК

Потапов Иван Романович

_____

Ставрополь 2026

Цель работы: реализовать решение задачи из предметной области в соответствии с вариантами (пп. 1.4) с использованием паттернов из списка: «Адаптер», «Мост».

Ход работы:

Вариант проекта – игра «Собиратель монет» на pygame.

1. Адаптер

Находится в coin_collector/input_adapter.py, классы PygameInputAdapter и InputFrame.

Задача: преобразовать специфические события Pygame в небольшую, удобную для игровой логики структуру ввода.

Как работает: адаптер принимает очередь событий и состояние клавиш, распознаёт WASD и стрелки, а затем возвращает направления движения, именованные действия и признак запроса на выход.

```
adapter = PygameInputAdapter()
frame = adapter.read(pygame.event.get(), pygame.key.get_pressed())

if frame.movement == (1, 0):
    print("игрок движется вправо")
if "confirm" in frame.pressed:
    print("нажата клавиша выбора")
if frame.quit_requested:
    print("запрошено закрытие игры")
```

Игровая логика получает объект с понятными действиями, а не должна самостоятельно сопоставлять клавиши:

```
@dataclass(frozen=True)
class InputFrame:
    movement: tuple[int, int] = (0, 0)
    pressed: frozenset[str] = frozenset()
    quit_requested: bool = False
```

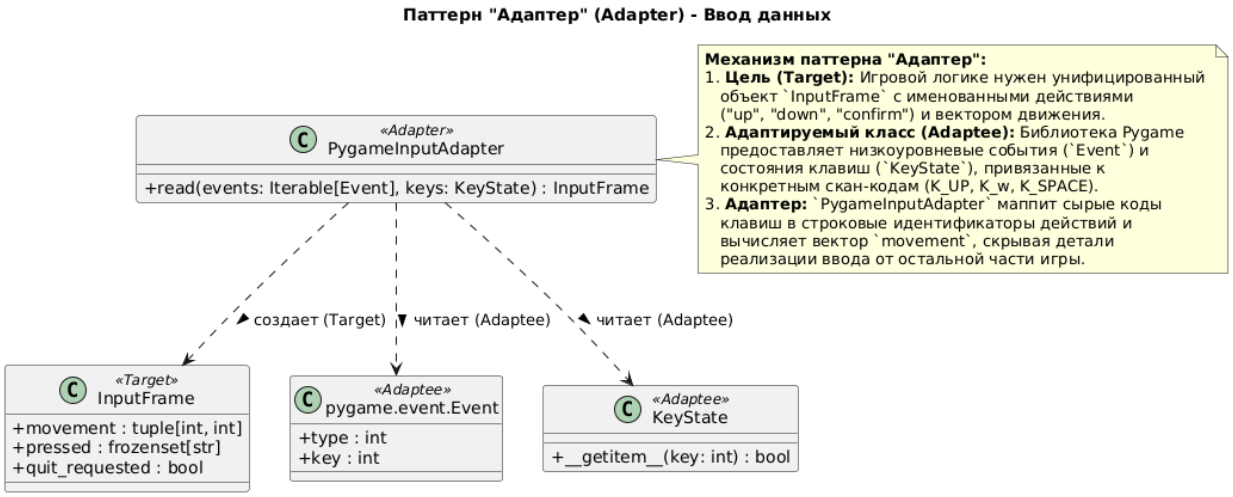


Рисунок 1. Диаграмма паттерна "Адаптер"

Таким образом, в игре поддерживаются два набора клавиш, но игровому циклу не нужно знать, каким именно способом игрок подал действие. В будущем при добавлении нового устройства ввода можно создать другой адаптер, который возвращает тот же InputFrame, и оставить обработку игры без изменений.

2. Мост

Находится в coin_collector/rendering.py, абстракция DrawingAPI и реализация PygameDrawingAPI; сущности используют этот интерфейс в методах draw().

Задача: отделить описание внешнего вида игровых объектов от низкоуровневых вызовов конкретной графической библиотеки.

Как работает: DrawingAPI задаёт набор операций рисования, а PygameDrawingAPI выполняет их вызовами pygame.draw. Персонаж, монета и опасность описывают изображение через операции этого интерфейса.

```
class DrawingAPI(ABC):
    @abstractmethod
    def circle(self, color: Color, center: Point, radius: int) -> None:
        raise NotImplementedError

    @abstractmethod
    def rectangle(self, color: Color, rect: pygame.Rect, radius: int = 0) ->
None:
        raise NotImplementedError
```

Конкретная реализация соединяет интерфейс с поверхностью Pygame:

```
class PygameDrawingAPI(DrawingAPI):
    def __init__(self, surface: pygame.Surface) -> None:
        self.surface = surface

    def circle(self, color: Color, center: Point, radius: int) -> None:
        pygame.draw.circle(self.surface, color, center, radius)
```

Сущность рисует себя через абстракцию, не обращаясь напрямую к pygame.draw:

```
def draw(self, drawing: DrawingAPI) -> None:
    r = self.rect
    drawing.rectangle((196, 65, 84), r, 10)
    drawing.circle((255, 255, 255), (r.centerx - 9, r.centery - 2), 5)
    drawing.circle((255, 255, 255), (r.centerx + 9, r.centery - 2), 5)
```

Приложение передаёт сущностям объект-мост, связанный с текущей поверхностью:

```
self.drawing = PygameDrawingAPI(self.screen)
self.level.player.draw(self.drawing)
```

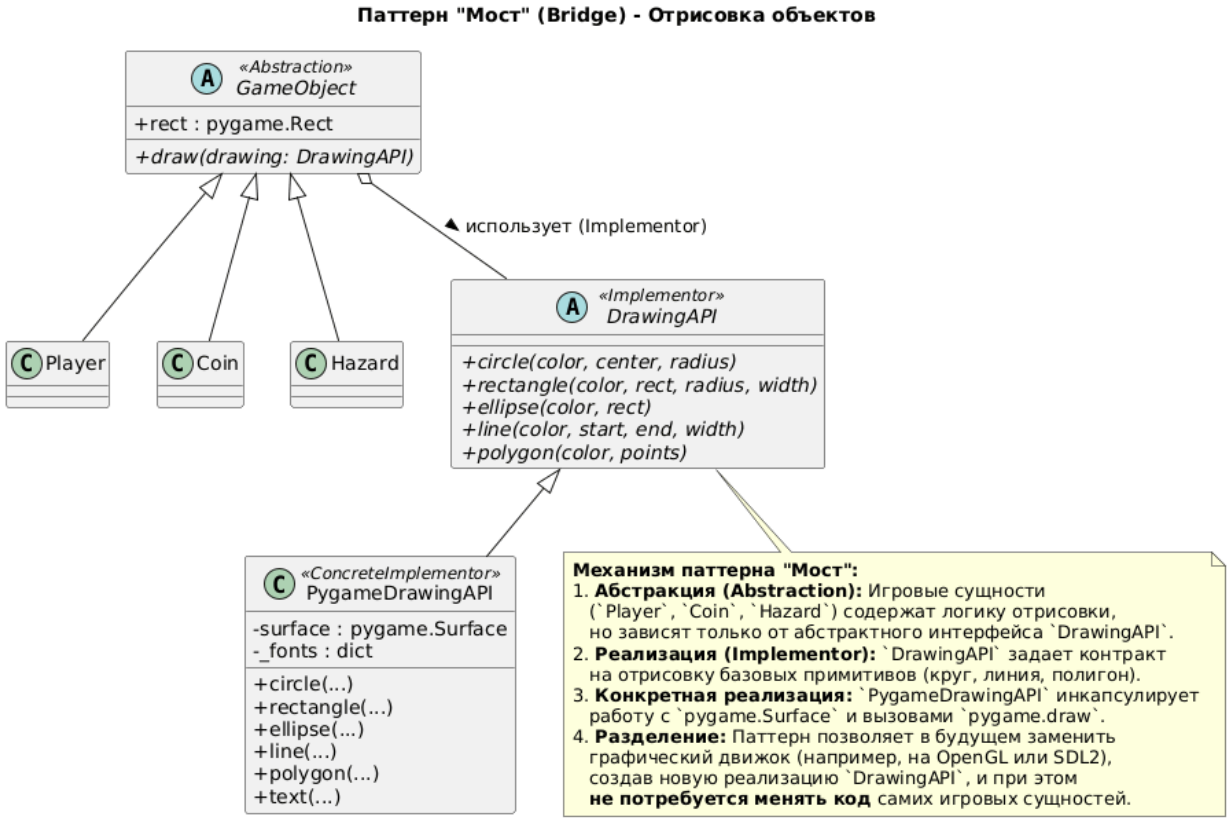


Рисунок 2. Диаграмма паттерна "Мост"

Таким образом, логика формы и анимации находится у игровых сущностей, а техническая реализация примитивов — в графическом адаптере. Изменить отрисовку или подменить её тестовой реализацией можно без переписывания моделей объектов.

Вывод:

Проделав работу, реализовал решение задачи из предметной области в соответствии с вариантами (пп. 1.4) с использованием паттернов «Адаптер» и «Мост».